National Academy of Science and Technology

Affiliated to Pokhara University

Accredited by University Grants Commission (UGC), Nepal 2022

Uttar-Behadi-4, Dhangadhi, Kailali

A Lab Report of

Operating System

For the partial fulfillment of degree of Computer Application

Awarded by Pokhara University

Submitted to

Mr. Kapil Dev Pant

Lecturer, Department of Computer Applications

Submitted by

Gajendra Awasthi (24530017)

Feb 2026

…… ……

Submission Date Signature

Table of Contents

Lab 1: Windows Installation

Objectives

Students will create a bootable media and install Windows.

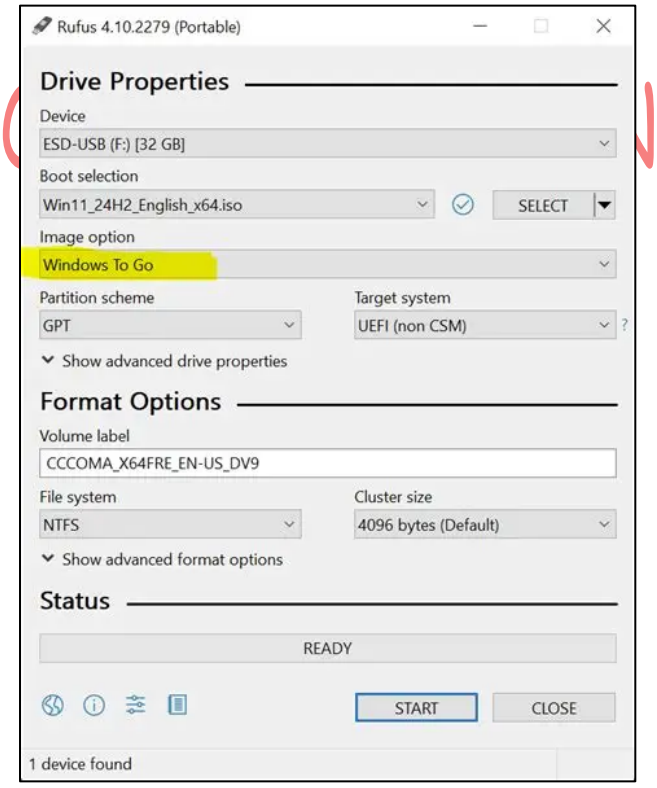
Requirements

- Windows ISO file
- USB drive (min 8 GB)
- Rufus or media creation tools
- VirtualBox/VMware

Procedures

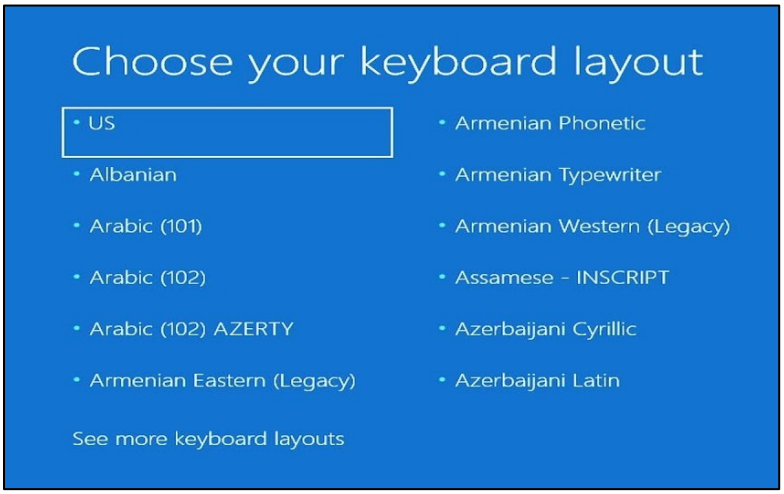
1. Download Windows ISO and Rufus.

2. Open Rufus -> Select ISO file -> Choose partition scheme (MBR/GPT) -> Start.

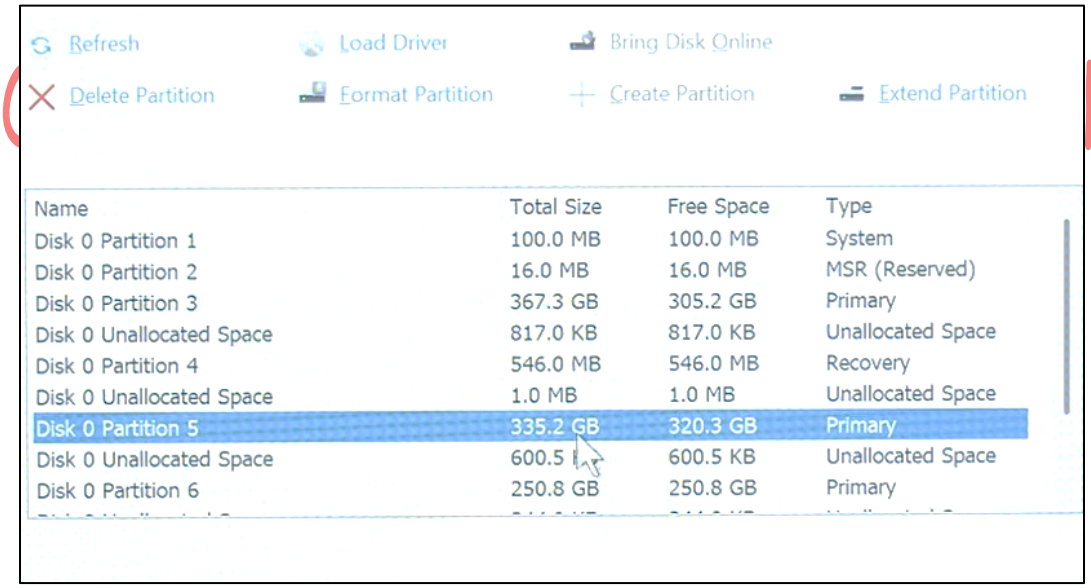


3. Boot from USB and begin Windows installation.

4. Select language, time, keyboard layout.



5. Choose installation type: Update or custom.
6. Partition disk (NTFS file system recommended).



7. Continue installation and restart system.
8. Complete initial setup.

Questions:

1. **Why NTFS instead of FAT32?**

   NTFS is preferred over FAT32 mainly because it's more powerful, secure, and suitable for modern systems. Here are the key reasons:

   - **Larger file and partition support**
     - **FAT32**: Maximum file size is **4 GB**, and partition size is limited.
     - **NTFS**: Supports very large files and volumes (terabytes and beyond).
   - **Security features**
     - **NTFS** supports file and folder permissions (ACLs), encryption (EFS), and auditing.
     - **FAT32** has **no built-in security**—any user can access any file.
   - **Reliability and recovery**
     - **NTFS** uses a **journaling system**, which helps prevent data corruption and allows recovery after crashes or power failures.
     - **FAT32** is more prone to corruption.
   - **Performance and efficiency**
     - **NTFS** handles large numbers of files more efficiently and reduces fragmentation compared to FAT32.
   - **Advanced features**
     - NTFS supports **compression, disk quotas, hard links, symbolic links**, and sparse files.
     - FAT32 lacks these capabilities.
   - **Modern OS compatibility**
     - Windows uses NTFS as the default for system drives. Many system features **require NTFS** to function properly.

2. **What is difference between MBR and GPT?**

Feature	MBR (Master Boot Record)	GPT (GUID Partition Table)
Maximum disk size	Up to 2 TB	Supports > 2 TB (up to ~9.4 ZB)
Maximum partitions	4 primary partitions (or 3 primary + 1 extended)	128 partitions (Windows default)
Partition identification	Uses partition IDs	Uses GUIDs (Globally Unique Identifiers)
Boot mode support	Legacy BIOS	UEFI (required for GPT boot)
Data reliability	Single partition table, prone to corruption	Primary + backup partition tables for recovery
Security	No built-in security	Supports Secure Boot
Boot data location	Stored in first sector of disk	Stored at beginning and end of disk
Compatibility	Older systems and OS	Modern systems and OS
Error detection	None	CRC32 checksums for integrity
Typical usage	Older PCs, legacy systems	Modern PCs, large disks, UEFI systems

3. **What is difference between upgrade and custom installations?**
   The major difference between upgrade and custom installations are shown on table below,

Feature	Upgrade Installation	Custom Installation
Purpose	Updates an existing OS to a newer version	Installs a **fresh OS**
Existing OS required	**Yes**	**No**
User files	**Kept**	**Deleted** (unless backed up)
Installed programs	**Retained**	**Removed**
System settings	Mostly preserved	Reset to default
Disk formatting	Not required	Often required / optional
Risk of errors	Higher (old issues may remain)	Lower (clean system)
Installation time	Faster	Slower
Best used when	System is stable and compatible	System is slow, corrupted, or infected

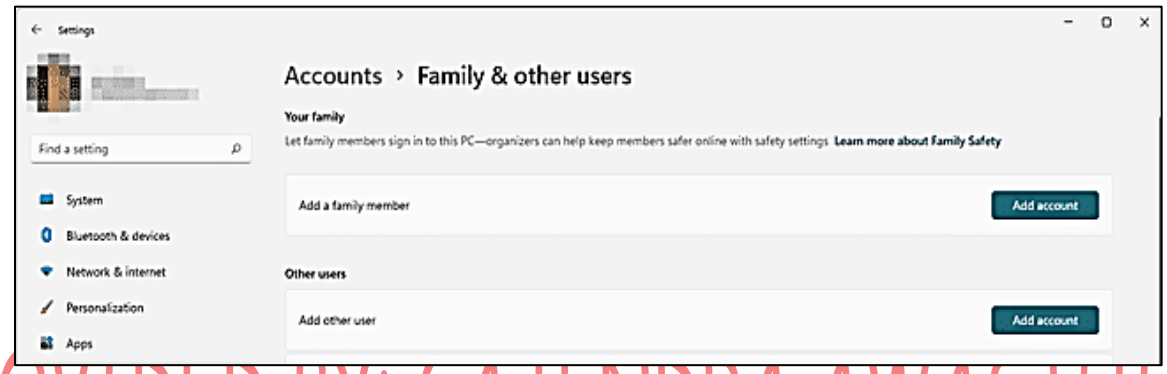
Lab 2: User Account and Application Management

Objectives

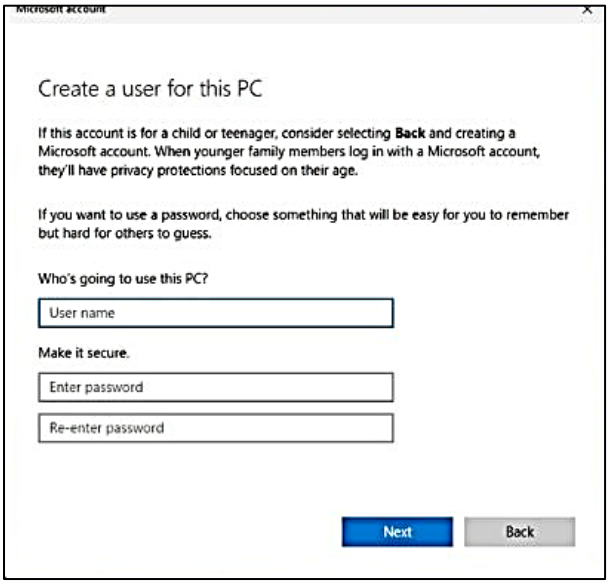
Students will learn to create and manage user accounts and roles in windows. They will later install, update and remove application in Windows.

Procedures

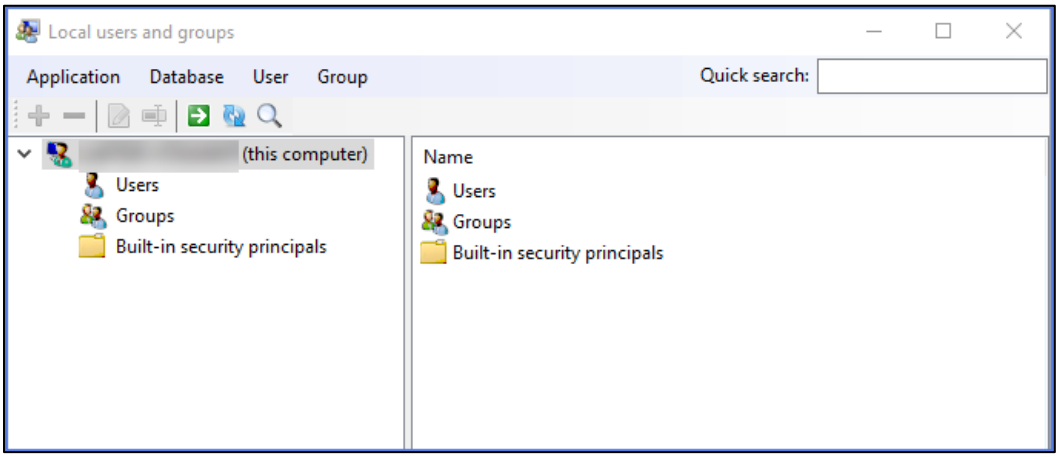
1. Create a local user via Settings -> Accounts-> Family and other users.



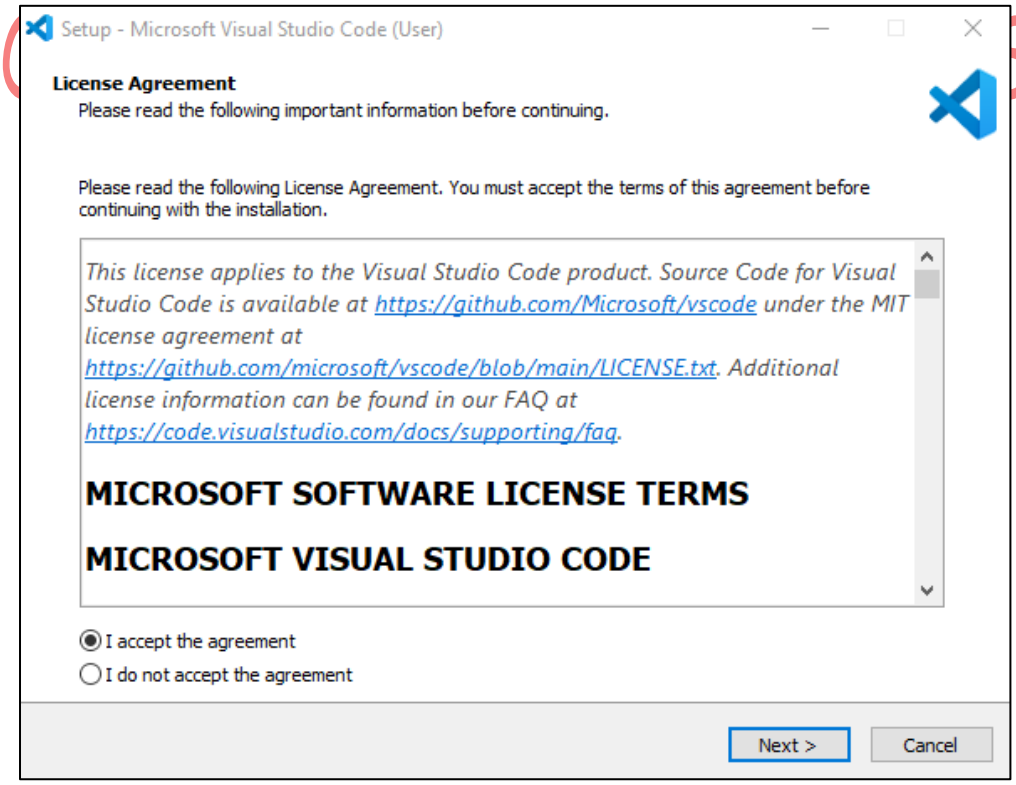
2. Create a local user via control panel -> User Accounts.



3. Use Computer Management -> Local Users and Groups.
    a. Examine built-in accounts (Administrator, Guest).
    b. Enable/disable accounts, reset passwords, and change roles.

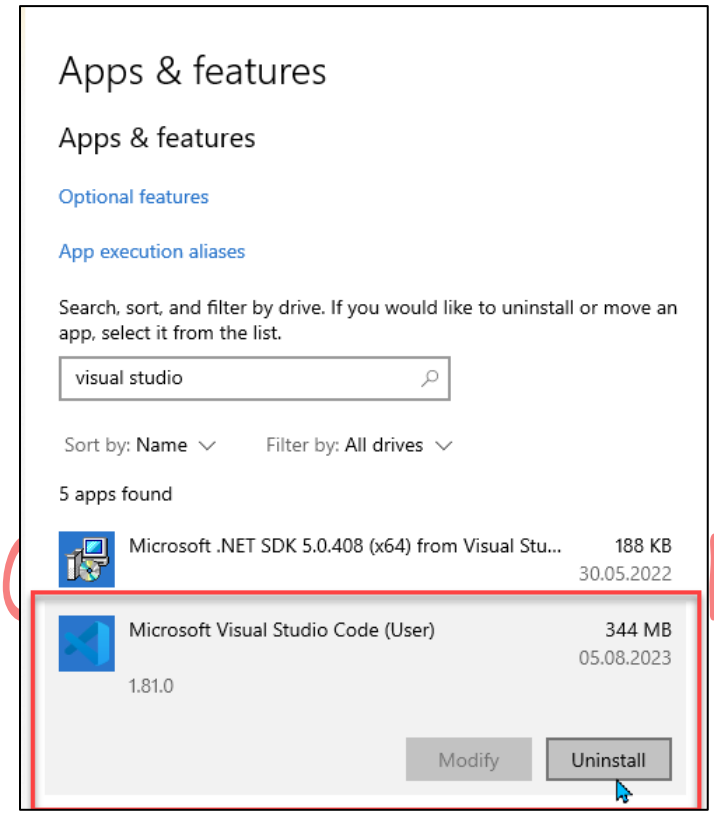


4. Install an Application (VS code)



5. Verify installation by launching app.

6. Remove application: Control Panel -> Programs and Features -> Uninstall.



Questions:

1. **Why are the Administrator disabled by default?**
   The Administrator account is disabled by default to enhance system security because it has full control over the operating system. If enabled, it could be easily exploited by malware or unauthorized users, leading to serious security breaches. Instead, modern operating systems use User Account Control (UAC), which allows users to perform administrative tasks only when necessary, thereby reducing the risk of accidental or malicious system changes

2. **What are security concerns with Guest Accounts?**
   Guest accounts pose significant security risks because they allow access to the system without proper authentication or accountability. Such accounts usually have no passwords and limited tracking, making it difficult to monitor user activity. Unauthorized users can

misuse system resources, access shared files, or introduce malware, which can compromise system integrity and data security. For these reasons, guest accounts are disabled by default in most modern operating systems.

3. **What is difference between uninstalling and disabling application?**
   Uninstalling an application completely removes it from the system, including its main program files, and frees up disk space. Once uninstalled, the application cannot be used unless it is reinstalled. Disabling an application, on the other hand, only turns off the application temporarily while keeping its files and settings on the system. This allows the application to be re-enabled later without reinstalling, but it does not free disk space.

4. **How to delete residues of uninstalled application?**
   After uninstalling an application, some leftover files and registry entries may remain on the system. These residues can be removed by manually deleting leftover folders from locations such as Program Files and the AppData directory, cleaning temporary files using Disk Cleanup, and carefully removing related registry entries using the Registry Editor. Alternatively, third-party uninstaller tools can be used to automatically detect and remove all remaining components, ensuring a clean system.

Lab 3: OS Installation in Virtual Box

Objectives

Students will install a Linux distribution (Ubuntu) and configure basic system settings

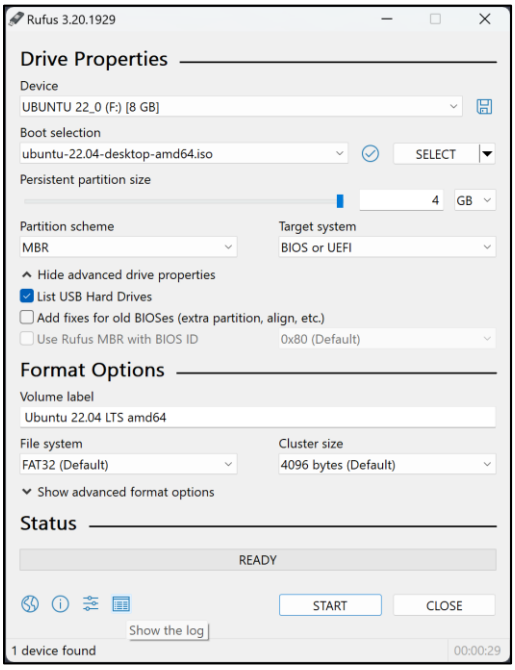
Requirements

- ISO file (Ubuntu 22.04 LTS or kali)
- VirtualBox/VMware
- System with 4GB+ RAM

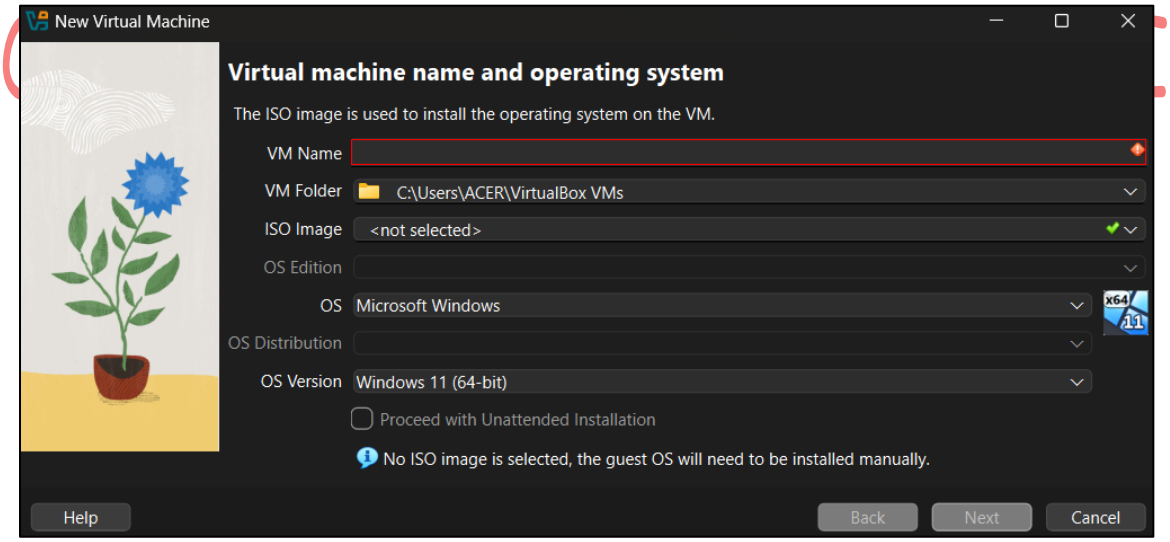
Procedures

1. Download Linux ISO from official website.

2. Create bootable media (Rufus).

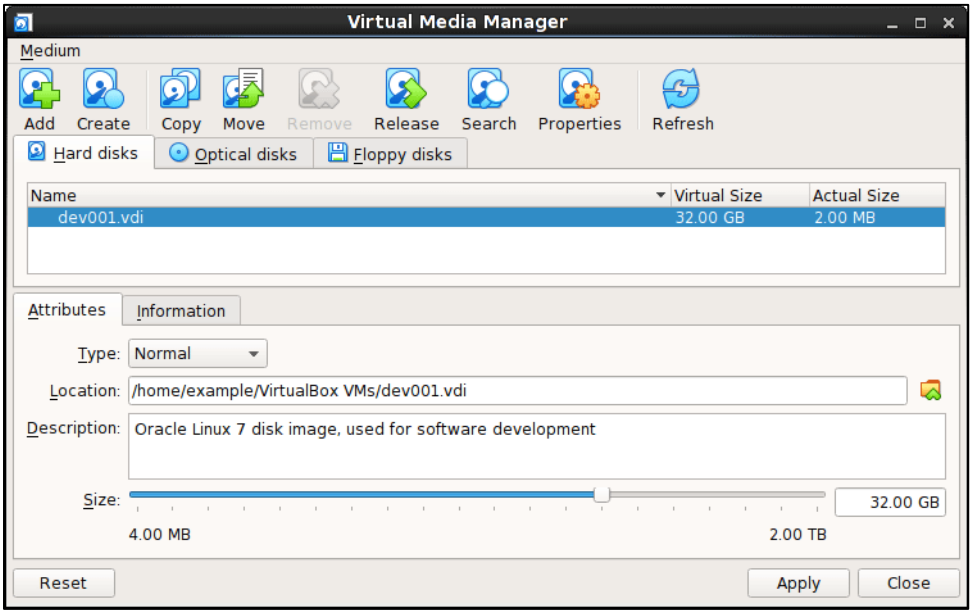


3. Boot system /VM from ISO.



4. Select Install Linux option.

5. Partition disk:
    a. Guided (use entire disk)
    b. Manual (/root/home, swap partitions)



6. Configure language, time zone, keyboard layout, and user account.
7. Finish installation and reboot.

Lab 4: User, Group and Application Management

Objectives

Students will learn to create, modify, and manage Linux users and groups. They will also lean to install, update, and remove applications using package managers.

Procedures

1. Check current user:
   whoami
   id
2. Add new user:
   sudo adduser student1
   sudo su
   exit

```
ubuntu@tryhackme:~$ whoami
ubuntu
ubuntu@tryhackme:~$ id
uid=1000(ubuntu) gid=1000(ubuntu) groups=1000(ubuntu),4(adm),20(dialout),
7(sudo),29(audio),30(dip),44(video),46(plugdev),117(netdev),118(lxd)
ubuntu@tryhackme:~$ sudo adduser cybgaz
info: Adding user `cybgaz' ...
info: Selecting UID/GID from range 1000 to 59999 ...
info: Adding new group `cybgaz' (1003) ...
info: Adding new user `cybgaz' (1003) with group `cybgaz (1003)' ...
info: Creating home directory `/home/cybgaz' ...
info: Copying files from `/etc/skel' ...
New password:
BAD PASSWORD: No password supplied
Retype new password:
No password has been supplied.
passwd: Authentication token manipulation error
passwd: password unchanged
Try again? [y/N]
Changing the user information for cybgaz
Enter the new value, or press ENTER for the default
        Full Name []: cyb gaz
        Room Number []: 1
        Work Phone []: 123
        Home Phone []:
        Other []:
Is the information correct? [Y/n] y
```

3. Update repositories:

   Sudo apt update

```
ubuntu@tryhackme:~$ sudo apt update
Ign:1 https://ppa.launchpadcontent.net/mozillateam/ppa/ubuntu noble InRelease
Ign:2 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble InRelease
Ign:3 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease
Ign:4 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease
Ign:1 https://ppa.launchpadcontent.net/mozillateam/ppa/ubuntu noble InRelease
Ign:2 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble InRelease
Ign:3 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease
Ign:4 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease
Ign:5 http://security.ubuntu.com/ubuntu noble-security InRelease
Ign:1 https://ppa.launchpadcontent.net/mozillateam/ppa/ubuntu noble InRelease
Ign:5 http://security.ubuntu.com/ubuntu noble-security InRelease
Ign:2 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble InRelease
Ign:3 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease
Ign:4 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease
```

4. Install an application (firefox)

   sudo apt install firefox -y


```
ubuntu@tryhackme:~$ sudo apt install firefox -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Suggested packages:
  fonts-lyx
Recommended packages:
  xul-ext-ubufox
The following packages will be DOWNGRADED:
  firefox
0 upgraded, 0 newly installed, 1 downgraded, 0 to remove and 0 not upgraded.
```

5. Verify installation

   code –version

6. Remove application:

   sudo apt remove code

```
ubuntu@tryhackme:~code -versionon
Command 'code' not found, but can be installed with:
sudo snap install code
ubuntu@tryhackme:~$ sudo apt remove code
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done

No apt package "code", but there is a snap with that name.
Try "snap install code"

E: Unable to locate package code
```


7. Remove unnecessary packages:

   sudo apt autoremove

```
ubuntu@tryhackme:~$ sudo apt autoremove
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
```

Lab 5: Command line interface in Linux

Objectives

Execution of basic commands used in Unix/Linux system.

Description

Open terminal. The terminal shows username and the prompt ($), after the prompt start the commands with different options.

The syntax of commands [command] [-options] [arg1, arg2…].

- **pwd** : to see the current or working directory
- **mkdir** : make a new directory
- **ls** : to see the list of file and directory
- **ls** -a: to see all file (hidden also)
- **ls** -l : to see the default permission of files
- **cd** <directory name>: to move inside folder **cd ..** get out of the directory
- **cat <filename>** : read content of filename
- **rm <filename> :** to remove file
- **rmdir <dirname> :** to remove directory

PROVIDED BY: GAJENDRA AWASTHI

```
ubuntu@tryhackme:~$ pwd
/home/ubuntu
ubuntu@tryhackme:~$ mkdir gajjubhai
ubuntu@tryhackme:~$ ls
Desktop    Downloads  Pictures  Templates  gajjubhai  projects
Documents  Music      Public    Videos     logs       snap
ubuntu@tryhackme:~$ ls -a
.                  .bashrc       .local                    .themes     Music      projects
..                 .cache        .logs                     .viminfo    Pictures   snap
.Xauthority        .config       .profile                  .vnc        Public
.Xresources        .dbus         .python_history           .wget-hsts  Templates
.apport-ignore.xml .gnupg        .research                 Desktop     Videos
.bash_history      .hidden_clue  .ssh                      Documents   gajjubhai
.bash_logout       .icons        .sudo_as_admin_successful Downloads   logs
ubuntu@tryhackme:~$ cd gajjubhai
ubuntu@tryhackme:~/gajjubhai$ touch gajju.txt
ubuntu@tryhackme:~/gajjubhai$ ls
gajju.txt
ubuntu@tryhackme:~/gajjubhai$ cat gajju.txt
ubuntu@tryhackme:~/gajjubhai$ rm gajju.txt
ubuntu@tryhackme:~/gajjubhai$ ls
ubuntu@tryhackme:~/gajjubhai$ cd ..
ubuntu@tryhackme:~$ ls
Desktop    Downloads  Pictures  Templates  gajjubhai  projects
Documents  Music      Public    Videos     logs       snap
ubuntu@tryhackme:~$ rmdir gajjubhai
ubuntu@tryhackme:~$ ls
Desktop    Downloads  Pictures  Templates  logs      snap
Documents  Music      Public    Videos     projects
```

```
ubuntu@tryhackme:~$ ls -l
total 44
drwxr-xr-x 2 ubuntu ubuntu 4096 Feb 27  2022 Desktop
drwxr-xr-x 6 ubuntu ubuntu 4096 Dec 11 12:45 Documents
drwxr-xr-x 2 ubuntu ubuntu 4096 Feb 16  2024 Downloads
drwxr-xr-x 2 ubuntu ubuntu 4096 Feb 27  2022 Music
drwxr-xr-x 2 ubuntu ubuntu 4096 Feb 27  2022 Pictures
drwxr-xr-x 2 ubuntu ubuntu 4096 Feb 27  2022 Public
drwxr-xr-x 2 ubuntu ubuntu 4096 Feb 27  2022 Templates
drwxr-xr-x 2 ubuntu ubuntu 4096 Feb 27  2022 Videos
drwxr-xr-x 4 ubuntu ubuntu 4096 Dec 11 12:29 logs
drwxr-xr-x 5 ubuntu ubuntu 4096 Dec 11 12:29 projects
drwx------ 3 ubuntu ubuntu 4096 Sep 12  2024 snap
```

Questions:

Q1. Display your present working directory.

The pwd command is used to display the full path of the current directory in which the user is working.

Q2. Display Calander of your birthday. Also check various options.

The cal command is used to display a calendar in Linux/Unix systems. To display the calendar of your birthday month, you can specify the month and year. Various options with the cal command allow viewing a full year, a specific month, or highlighting the current date.

Different Option Included In cal command are , cal -3 (Display previous, current, and next month) , cal -w (Display week numbers).

Q3. Display current date. Also check various options.

The date command is used to display the current system date and time in Linux/Unix.

The Various Function date command included are,

- date +"%d-%m-%Y" : Display only the current date
- date +"%A, %d %B %Y" : Display date in words
- date +"%T" : Display current time only
- date -u : Display UTC date and time
- date -d "yesterday" : Display yesterday's date
- date -d "tomorrow" : Display tomorrow's date.

Q3. Check whether the following commands are built-in or external commands:

pwd, cd, man, cat, file, echo, who, ls.

The following commands are categorized according to type in given table below,

Command	Type	Explanation
pwd	**Built-in**	Provided by the shell to display the current directory
cd	**Built-in**	Changes directory; must be built-in to affect shell
man	**External**	Executable program used to display manuals
cat	**External**	Executable program to display file contents
file	**External**	Program used to determine file type
echo	**Built-in**	Implemented by the shell to print output
who	**External**	Executable program showing logged-in users
ls	**External**	Executable program to list directory contents

Q4. Check "help", "man" and explain their purpose.

The **help** command is used to display information about **shell built-in commands**. It provides a brief explanation of what a built-in command does along with its available options and syntax. Since built-in commands are part of the shell itself, help is faster and mainly useful for quick reference.

The **man** (manual) command is used to display **detailed documentation** for both built-in and external commands. It shows complete manuals including command description, syntax, options, examples, and related commands. The man pages are more comprehensive and are used when in-depth understanding of a command is required.

Q5. Create a directory using your name.

To create a directory with your name in Linux/Unix, you can use the mkdir command. For example, if your my is Gajendra i use command, " mkdir Gajendra " . Here is how this command works,

mkdir = "make directory"

Gajendra = name of the directory to create

Q6. Create two files by the name f1.txt, f2.txt.

To create two files named f1.txt and f2.txt in Linux/Unix, you can use the **touch** command:

touch f1.txt f2.txt. Here how this command works,

touch : is used to **create empty files** or update the timestamp of existing files.

f1.txt f2.txt : specifies the names of the files to create.

Q7. Analyze the default permission of both files.

To analyze the default permissions of the files f1.txt and f2.txt, you use the ls -l command:

ls -l f1.txt f2.txt. by entering this command we can able to view the permission output like this,

-rw-r--r-- 1 username group 0 Jan 24 10:00 f1.txt

-rw-r--r-- 1 username group 0 Jan 24 10:00 f2.txt

Lets analyze how this command works,

Symbol	Meaning
-	Indicates it's a **regular file** (not a directory or link)
rw-	**Owner permissions**: read (r) and write (w), no execute (-)
r--	**Group permissions**: read only
r--	**Others permissions**: read only

Q8. Delete file, then delete directory.

To delete a file and then a directory in Linux/Unix, you can use the following commands:

1. Delete a file

To delete a file (e.g., f1.txt):

rm f1.txt

- rm = remove file
- You can delete multiple files at once:

rm f1.txt f2.txt

2. Delete a directory

To delete a directory (e.g., Alex):

rmdir Alex

- rmdir removes empty directories only.

If the directory is not empty, use:

rm -r Alex

- -r = recursive, deletes the directory and all its contents

Verify deletion

After deleting, check the current directory:

- ls

The files and directory should no longer appear.

Q9. Display all the files and directory including hidden files in your system in home directory.

To **display all files and directories, including hidden ones, in your home directory**, you can use the **ls** command with the -a option:

ls -a ~

Here how this command runs,

- ls → lists files and directories
- -a → includes **hidden files** (those starting with .)
- ~ → represents your **home directory**

Here is how the command output looks like,

```
.               .bashrc       Documents     f1.txt
..              .profile      Downloads     f2.txt
.bash_history   .config       Pictures      Gajendra
```

PROVIDED BY: GAJENDRA AWASTHI

Lab 6: CPU Scheduling Simulation using FCFS without arrival time

Objectives:

Simulate basic CPU algorithm

Calculate turnaround and waiting times

Algorithm:

1. Start
2. Input number of processes n.
3. Input burst time BT[i] for each process P[i] for i = 1...n.
4. Set current_time = 0.
5. For i = 1 to n (processes in the given order):
    a. StartTime[i] = current_time.
    b. FinishTime[i] = StartTime[i] + BT[i].
    c. WaitingTime[i] = StartTime[i] (since arrival = 0).
    d. TurnAroundTime[i] = FinishTime[i] (since arrival = 0).
    e. current_time = FinishTime[i].
6. Compute Average Waiting Time = sum(WaitingTime[i]).
7. Compute Average Turnaround Time = sum(TurnAroundTime[i]).
8. Output Start, Finish, Waiting, Turnaround times and the averages.
9. Stop

Pseudocode:

```
Input n
for i = 1 to n:
   input BT[i]

current_time = 0
total_WT = 0
total_TAT = 0
for i = 1 to n:
   StartTime[i] = current_time
   FinishTime[i] = StartTime[i] + BT[i]
   WaitingTime[i] = StartTime[i]          // arrival assumed 0
   TurnAroundTime[i] = FinishTime[i]      // arrival assumed 0
   total_WT += WaitingTime[i]
   total_TAT += TurnAroundTime[i]
   current_time = FinishTime[i]

avg_WT = total_WT / n
avg_TAT = total_TAT / n

print results
```

Program:

```
#include <stdio.h>

int main()
{
    int burst[10], pid[10];
    int start[10], finish[10], wait[10], turn[10];
    int i, n;

    float totalwait = 0.0, totalturn = 0.0;

    printf("Name: Gajendra Awasthi \n");
    printf("Enter number of processes: ");
    scanf("%d", &n);

    // Input Burst Times
    for (i = 0; i < n; i++)
    {
        pid[i] = i + 1;
        printf("Enter Burst time of Process %d: ", i + 1);
        scanf("%d", &burst[i]);
    }

    // FCFS Scheduling (All arrival = 0)
    start[0] = 0;
    finish[0] = burst[0];

    for (i = 1; i < n; i++)
    {
        start[i] = finish[i - 1];
        finish[i] = start[i] + burst[i];
    }

    // Calculate turnaround and waiting time
    for (i = 0; i < n; i++)
    {
        turn[i] = finish[i]; // arrival = 0
        wait[i] = turn[i] - burst[i];

        totalwait += wait[i];
        totalturn += turn[i];
    }

    // Output
```

```
    printf("\nProcess\tBurst\tStart\tFinish\tTurn\tWait\n");
    for (i = 0; i < n; i++)
    {
        printf("P%d\t%5d\t%5d\t%6d\t%4d\t%4d\n",
               pid[i], burst[i], start[i], finish[i], turn[i], wait[i]);
    }

    printf("\nAverage Waiting Time: %.2f\n", totalwait / n);
    printf("Average Turnaround Time: %.2f\n", totalturn / n);

    return 0;
}
```

Output

```
Name: Gajendra Awasthi
Enter number of processes: 5
Enter Burst time of Process 1: 5
Enter Burst time of Process 2: 10
Enter Burst time of Process 3: 15
Enter Burst time of Process 4: 20
Enter Burst time of Process 5: 25

Process Burst   Start   Finish  Turn    Wait
P1      5       0       5       5       0
P2      10      5       15      15      5
P3      15      15      30      30      15
P4      20      30      50      50      30
P5      25      50      75      75      50

Average Waiting Time: 20.00
Average Turnaround Time: 35.00
```

Lab 7: CPU Scheduling Simulation using FCFS with arrival time

Objectives:

Simulate basic CPU algorithm

Calculate turnaround and waiting times

Algorithm:

1. Start
2. Read the number of processes n
3. For each process i
    a. Input Arrival Time and Burst Time
    b. Assign a Process ID
4. Sort all processes in ascending order of Arrival Time
   (If arrival times are equal, smaller PID comes first)
5. Set start[0] = arrival[0]
6. Compute finish[0] = start[0] + burst[0]
7. For each process i = 1 to n-1
        If previous finish time < arrival time
            CPU is idle → start[i] = arrival[i]
        Else
            start[i] = finish[i - 1]
        Compute finish[i] = start[i] + burst[i]
8. For each process

    Turnaround time = finish[i] - arrival[i]

    Waiting time = turnaround - burst[i]
9. Compute Average Turnaround Time and Average Waiting Time
10. Display the results
11. Stop

Pseudocode:

```
BEGIN
   INPUT n
   FOR i = 1 TO n DO
       pid[i] = i
       INPUT arrival[i], burst[i]
   END FOR
   // Sort by arrival time
   FOR i = 1 TO n-1 DO
      FOR j = i+1 TO n DO
         IF arrival[j] < arrival[i] THEN
            SWAP arrival[i], arrival[j]
            SWAP burst[i], burst[j]
            SWAP pid[i], pid[j]
         END IF
      END FOR
   END FOR
   start[1] = arrival[1]
   finish[1] = start[1] + burst[1]

   FOR i = 2 TO n DO
      IF finish[i-1] < arrival[i] THEN
         start[i] = arrival[i]
      ELSE
         start[i] = finish[i-1]
      END IF

      finish[i] = start[i] + burst[i]
   END FOR

   FOR i = 1 TO n DO
      turn[i] = finish[i] - arrival[i]
      wait[i] = turn[i] - burst[i]
   END FOR

   COMPUTE average waiting time
   COMPUTE average turnaround time

   DISPLAY all process values
END
```

Program:

```
#include <stdio.h>

int main()
{
   int arrival[10], burst[10], pid[10];
   int start[10], finish[10], wait[10], turn[10];
   int i, j, n;

   float totalwait = 0.0, totalturn = 0.0;

   printf("Name: Gajendra Awasthi \n");
   printf("Enter number of processes: ");
   scanf("%d", &n);

   // Input
   for (i = 0; i < n; i++)
   {
      pid[i] = i + 1;
      printf("Enter Arrival and Burst time of Process %d: ", i + 1);
      scanf("%d %d", &arrival[i], &burst[i]);
   }

   // SORT BY ARRIVAL TIME (if all arrival same then → lower PID first)
   for (i = 0; i < n - 1; i++)
   {
      for (j = i + 1; j < n; j++)
      {
         if (arrival[j] < arrival[i] ||
            (arrival[j] == arrival[i] && pid[j] < pid[i]))
         {
            // Swap arrival
            int temp = arrival[i];
            arrival[i] = arrival[j];
            arrival[j] = temp;

            // Swap burst
            temp = burst[i];
            burst[i] = burst[j];
            burst[j] = temp;

            // Swap process ID
            temp = pid[i];
            pid[i] = pid[j];
            pid[j] = temp;
         }
```

```
        }
    }

    // FCFS Scheduling Calculations

    start[0] = arrival[0];
    finish[0] = start[0] + burst[0];

    for (i = 1; i < n; i++)
    {
        if (finish[i - 1] < arrival[i])
            start[i] = arrival[i]; // CPU idle
        else
            start[i] = finish[i - 1];

        finish[i] = start[i] + burst[i];
    }

    // Calculate  turnaround and wait time
    for (i = 0; i < n; i++)
    {

        turn[i] = finish[i] - arrival[i];
        wait[i] = turn[i] - burst[i];

        totalwait += wait[i];
        totalturn += turn[i];
    }

    // Output

    printf("\nProcess\tArrival\tBurst\tStart\tFinish\tTurn\twait\n");
    for (i = 0; i < n; i++)
    {
        printf("P%d\t%7d\t%5d\t%5d\t%6d\t%4d\t%4d\n",
               pid[i], arrival[i], burst[i], start[i], finish[i], turn[i], wait[i]);
    }

    printf("\nAverage Waiting Time: %.2f\n", totalwait / n);
    printf("Average Turnaround Time: %.2f\n", totalturn / n);

    return 0;
}
```

Output

```
Name: Gajendra Awasthi
Enter number of processes: 5
Enter Arrival and Burst time of Process 1: 5
10
Enter Arrival and Burst time of Process 2: 15
20
Enter Arrival and Burst time of Process 3: 25
30
Enter Arrival and Burst time of Process 4: 35
40
Enter Arrival and Burst time of Process 5: 45
50

Process Arrival Burst   Start   Finish  Turn    wait
P1        5    10       5       15    10       0
P2       15    20      15       35    20       0
P3       25    30      35       65    40      10
P4       35    40      65      105    70      30
P5       45    50     105      155   110      60

Average Waiting Time: 20.00
Average Turnaround Time: 50.00
```

WASTHI

Lab 8: CPU Scheduling Simulation (SJF)

Objectives

Simulate

Calculate turnaround and waiting times

Algorithm

1. Start
2. Input number of processes n
3. Input Arrival Time & Burst Time for each process
4. Set all processes as not completed
5. Set current_time = 0
6. While not all processes completed:
    a. Among all processes that have arrived (arrival[i] <= current_time) and not completed
    b. choose the one with the smallest burst time
    If no process has arrived
      current_time++
    Else
      Set start time
      finish time = start + burst
      mark process as completed
      move current_time to finish time
7. Calculate Waiting & Turnaround Time
8. Display results
9. Stop

Pseudocode

```
BEGIN
   INPUT n
   FOR each process i
      INPUT arrival[i], burst[i]
      completed[i] = false
   END FOR
   current_time = 0
   finished = 0
   WHILE finished < n DO
      min_burst = ∞
      selected = -1
      FOR each process i
         IF arrival[i] <= current_time AND completed[i] = false THEN
            IF burst[i] < min_burst THEN
               min_burst = burst[i]
               selected = i
            END IF
         END IF
      END FOR

      IF selected = -1 THEN
         current_time = current_time + 1
      ELSE
         start[selected] = current_time
         finish[selected] = current_time + burst[selected]
         completed[selected] = true
         current_time = finish[selected]
         finished = finished + 1
      END IF
   END WHILE

   FOR each process i
      turn[i] = finish[i] - arrival[i]
      wait[i] = turn[i] - burst[i]
   END FOR

   DISPLAY results
END
```

Program:

```
#include <stdio.h>

int main()
{
   int pid[20], arrival[20], burst[20];
   int start[20], finish[20], wait[20], turn[20];
   int completed[20] = {0};
   int n, i, time = 0, completed_count = 0;

   float total_wait = 0, total_turn = 0;

   printf("Your name: Gajendra Awasthi \n");
   printf("Enter number of processes: ");
   scanf("%d", &n);

   for (i = 0; i < n; i++)
   {
      pid[i] = i + 1;
      printf("Enter Arrival and Burst time for Process %d: ", i + 1);
      scanf("%d %d", &arrival[i], &burst[i]);
   }

   // SJF  ko main logic
   while (completed_count < n)
   {
      int idx = -1;
      int min_burst = 99999;

      // find the shortest job among arrived processes
      for (i = 0; i < n; i++)
      {
         if (!completed[i] && arrival[i] <= time)
         {
            if (burst[i] < min_burst)
            {
               min_burst = burst[i];
               idx = i;
            }
         }
      }

      // If no process has arrived, CPU is idle → jump to next arrival
      if (idx == -1)
      {
         int next_arrival = 99999;
```

```
            for (i = 0; i < n; i++)
            {
                if (!completed[i] && arrival[i] < next_arrival)
                    next_arrival = arrival[i];
            }
            time = next_arrival;
            continue;
        }

        // Process selected → run it
        start[idx] = time;
        finish[idx] = time + burst[idx];
        wait[idx] = start[idx] - arrival[idx];
        turn[idx] = finish[idx] - arrival[idx];

        total_wait += wait[idx];
        total_turn += turn[idx];

        completed[idx] = 1;
        completed_count++;

        time = finish[idx];
    }

    // OUTPUT
    printf("\nProcess\tArrival\tBurst\tStart\tFinish\tWait\tTurn\n");
    for (i = 0; i < n; i++)
    {
        printf("P%d\t%7d\t%5d\t%5d\t%6d\t%4d\t%4d\n",
               pid[i], arrival[i], burst[i], start[i], finish[i], wait[i], turn[i]);
    }

    printf("\nAverage Waiting Time: %.2f\n", total_wait / n);
    printf("Average Turnaround Time: %.2f\n", total_turn / n);

    return 0;
}
```

Output

```
Your name: Gajendra Awasthi
Enter number of processes: 5
Enter Arrival and Burst time for Process 1: 5 10
Enter Arrival and Burst time for Process 2: 15 20
Enter Arrival and Burst time for Process 3: 25 30
Enter Arrival and Burst time for Process 4: 35 40
Enter Arrival and Burst time for Process 5: 45 50

Process Arrival Burst   Start   Finish  Wait    Turn
P1        5    10       5       15     0      10
P2       15    20      15       35     0      20
P3       25    30      35       65    10      40
P4       35    40      65      105    30      70
P5       45    50     105      155    60     110

Average Waiting Time: 20.00
Average Turnaround Time: 50.00
```

Lab 9: CPU Scheduling Simulation (Round Robin)

Objectives

Simulate basic CPU algorithm

Calculate turnaround and waiting times

Algorithm:

1. Start
2. Input number of processes n
3. Input burst time for each process
4. Input time slice ts
5. Initialize need[i] = burst[i], totaltime = 0, and calculate totalburst
6. While totalburst > 0
    a. For each process:
        i. If need[i] > ts, execute for ts
        ii. Else execute for remaining time
        iii. Update totaltime, need[i], wait[i], finish[i], turn[i]
7. Calculate average waiting, turnaround, and response time
8. Display results
9. Stop

Program:

```
#include <stdio.h>
// #include<conio.h>
void main()
{
   int start[10], burst[10], need[10], execution[10], wait[10], finish[10], turn[10];
   int i, ts, n, totaltime = 0, totalburst = 0;
   float totalwait = 0.0, totalturn = 0.0, totalresp = 0.0;
   float avgwait = 0.0, avgturn = 0.0, avgresp = 0.0;
   // clrscr();
   printf("Name: Gajendra Awasthi\n");
   printf("Enter number of processes: ");
   scanf("%d", &n);
   for (i = 0; i < n; i++)
   {
      printf("Enter process %d burst time: ", (i + 1));
      scanf("%d", &burst[i]);
```

```
}
printf("Enter time slice: ");
scanf("%d", &ts);
for (i = 0; i < n; i++)
{
   need[i] = burst[i];
   execution[i] = 0;
   wait[i] = 0;
   finish[i] = 0;
   turn[i] = 0;
   totalburst = totalburst + burst[i];
}
while (totalburst > 0)
{
   for (i = 0; i < n; i++)
   {
      if (execution[i] == 0)
      {
         start[i] = totaltime;
      }
      if (need[i] > ts)
      {
         execution[i] = execution[i] + ts;
         need[i] = need[i] - ts;
         totaltime = totaltime + ts;
         totalburst = totalburst - ts;
      }
      else
      {
         if (need[i] > 0)
         {
            execution[i] = execution[i] + need[i];
            totaltime = totaltime + need[i];
            wait[i] = totaltime - execution[i];
            finish[i] = wait[i] + burst[i];
            turn[i] = wait[i] + burst[i];
            totalburst = totalburst - need[i];
            need[i] = 0;
         }
      }
   }
}
printf("\n process| burst| start| wait| finish| turnaround \n");
for (i = 0; i < n; i++)
{
```

```
        printf("%7d %5d %5d %5d %4d %6d \n", (i + 1), burst[i], start[i], wait[i], finish[i], turn[i]);
    }
    for (i = 0; i < n; i++)
    {
        totalwait = totalwait + wait[i];
        totalturn = totalturn + turn[i];
        totalresp = totalresp + start[i];
    }
    avgwait = totalwait / n;
    avgturn = totalturn / n;
    avgresp = totalresp / n;
    printf("\n Average waiting time %f\n", avgwait);
    printf("\n Average turnaround time %f\n", avgturn);
    printf("\n Average response time %f\n", avgresp);
    // getch();
}
```

Output

```
Name:Gajendra Awasthi
Enter number of processes: 5
Enter process 1 burst time: 5
Enter process 2 burst time: 10
Enter process 3 burst time: 15
Enter process 4 burst time: 20
Enter process 5 burst time: 25
Enter time slice: 3

 process| burst| start| wait| finish| turnaround
      1     5     0    12   17     17
      2    10     3    32   42     42
      3    15     6    39   54     54
      4    20     9    48   68     68
      5    25    12    50   75     75

 Average waiting time 36.200001

 Average turnaround time 51.200001

 Average response time 6.000000
```

Lab 10: CPU Scheduling Simulation (Priority based)

Objectives

Simulate basic CPU algorithm

Analyse pre-emptive scheduling.

Program:

```
#include <stdio.h>
typedef struct
{
   int id, burst, pri, start, wait, finish;
} Process;
void swap(Process *a, Process *b)
{
   Process temp = *a;
   *a = *b;
   *b = temp;
}
int main()
{
   int n, i, j;
   float totalwait = 0, totalturn = 0;
   printf("Name: Gajendra Awasthi \n");
   printf("Enter total number of processes: ");
   scanf("%d", &n);

   Process p[n];

   for (i = 0; i < n; i++)
   {
      p[i].id = i + 1;
      printf("\nEnter Burst time and Priority of process %d: ", i + 1);
      scanf("%d %d", &p[i].burst, &p[i].pri);
   }

   // this steps will Sort processes by priority (higher priority first)
   for (i = 0; i < n - 1; i++)
   {
      for (j = i + 1; j < n; j++)
      {
```

```
            if (p[i].pri < p[j].pri)
            { // higher priority = larger value
                swap(&p[i], &p[j]);
            }
        }
    }

    // Calculate Start, Wait, Finish times
    p[0].start = 0;
    p[0].wait = 0;
    p[0].finish = p[0].burst;

    for (i = 1; i < n; i++)
    {
        p[i].start = p[i - 1].finish;
        p[i].wait = p[i].start;
        p[i].finish = p[i].start + p[i].burst;
    }

    // Print results
    printf("\nProcess | Burst | Priority | Start | Wait | Finish\n");
    printf("-----------------------------------------------\n");

    for (i = 0; i < n; i++)
    {
        printf("P%-6d | %-5d | %-8d | %-5d | %-4d | %-6d\n",
               p[i].id, p[i].burst, p[i].pri, p[i].start, p[i].wait, p[i].finish);
        totalwait += p[i].wait;
        totalturn += p[i].finish;
    }

    printf("\nAverage Waiting Time    = %.2f", totalwait / n);
    printf("\nAverage Turnaround Time = %.2f\n", totalturn / n);

    return 0;
}
```

Output

```
Name: Gajendra Awasthi
Enter total number of processes: 5

Enter Burst time and Priority of process 1: 5 10

Enter Burst time and Priority of process 2: 15 20

Enter Burst time and Priority of process 3: 25 30

Enter Burst time and Priority of process 4: 35 40

Enter Burst time and Priority of process 5: 45 50

Process | Burst | Priority | Start | Wait | Finish
-----------------------------------------------
P5      | 45    | 50       | 0     | 0    | 45
P4      | 35    | 40       | 45    | 45   | 80
P3      | 25    | 30       | 80    | 80   | 105
P2      | 15    | 20       | 105   | 105  | 120
P1      | 5     | 10       | 120   | 120  | 125

Average Waiting Time    = 70.00
Average Turnaround Time = 95.00
```

Lab 11: Page Replacement Simulation using FIFO

Objectives

Simulate basic page replacement using FIFO approach

Algorithm:

Step 1: Start the process.

Step 2: Declare the number of frames.

Step 3: Read the number of pages in the reference string.

Step 4: Read the page reference values.

Step 5: Initialize all frames as empty and set a pointer to the first frame.

Step 6: For each page in the reference string, check whether the page is already present in the frames.

- If present, it is a page hit (no replacement).

Step 7: If the page is not present (page fault):

- Replace the page that was loaded first (oldest page) in memory.
- Move the pointer to the next frame in circular order.

Step 8: Display the contents of frames after each page reference.

Step 9: Count and display the total number of page faults.

Step 10: Stop the process.

Program:

```
#include <stdio.h>

int main()
{
   int i, j, k = 0;
   int n, m;
   int rs[30], p[30];
   int index = 0;
   int flag;

   printf("Name: CYBGAZ\n");
   printf("FIFO Page Replacement Algorithm\n\n");

   printf("Enter number of frames: ");
   scanf("%d", &n);
```

```
printf("Enter reference string (end with -1): ");
i = 0;
while (1)
{
   scanf("%d", &rs[i]);
   if (rs[i] == -1)
      break;
   i++;
}
m = i;

/* Initialize frames */
for (i = 0; i < n; i++)
   p[i] = -1;

/* FIFO logic */
for (i = 0; i < m; i++)
{
   flag = 0;

   /* Check page hit */
   for (j = 0; j < n; j++)
   {
      if (p[j] == rs[i])
      {
         flag = 1;
         break;
      }
   }

   /* Page fault */
   if (flag == 0)
   {
      p[index] = rs[i];
      index = (index + 1) % n;
      k++;

      printf("\nPage fault for %d\n", rs[i]);
      for (j = 0; j < n; j++)
         printf("Frame %d: %d\n", j + 1, p[j]);
   }
   else
   {
      printf("\nPage hit for %d\n", rs[i]);
   }
```

```
    }

    printf("\nTotal number of page faults = %d\n", k);

    return 0;
}
```

Output:

```
Name: CYBGAZ
FIFO Page Replacement Algorithm

Enter number of frames: 5 3 5 6 3 3 5 6 4 3 6 8 4 -1
Enter reference string (end with -1):
Page fault for 3
Frame 1: 3
Frame 2: -1
Frame 3: -1
Frame 4: -1
Frame 5: -1

Page fault for 5
Frame 1: 3
Frame 2: 5
Frame 3: -1
Frame 4: -1
Frame 5: -1

Page fault for 6
Frame 1: 3
Frame 2: 5
Frame 3: 6
Frame 4: -1
Frame 5: -1

Page hit for 3

Page hit for 3

Page hit for 5

Page hit for 6

Page fault for 4
```

Lab 12: Page Replacement Simulation using OPT

Objectives

Simulate basic CPU algorithm

Calculate turnaround and waiting times

Program:

```
#include <stdio.h>


int main()
{
   int i, j, k, n, m;
   int rs[30], p[30];
   int fault = 0, flag, index, farthest;
   int pos[30];

   printf("Name: Gajendra\n");
   printf("Optimal Page Replacement Algorithm\n\n");

   printf("Enter number of frames: ");
   scanf("%d", &n);

   printf("Enter reference string (end with -1): ");
   i = 0;
   while (1)
   {
      scanf("%d", &rs[i]);
      if (rs[i] == -1)
         break;
      i++;
   }
   m = i;

   /* Initialize frames */
   for (i = 0; i < n; i++)
      p[i] = -1;

   /* Optimal replacement logic */
   for (i = 0; i < m; i++)
   {
      flag = 0;

      /* Check page hit */
```

```
for (j = 0; j < n; j++)
{
   if (p[j] == rs[i])
   {
      flag = 1;
      break;
   }
}

/* Page fault */
if (flag == 0)
{
   fault++;

   /* If empty frame exists */
   for (j = 0; j < n; j++)
   {
      if (p[j] == -1)
      {
         p[j] = rs[i];
         goto display;
      }
   }

   /* Find page to replace */
   for (j = 0; j < n; j++)
   {
      pos[j] = -1;
      for (k = i + 1; k < m; k++)
      {
         if (p[j] == rs[k])
         {
            pos[j] = k;
            break;
         }
      }
   }

   /* Find farthest future use */
   farthest = 0;
   index = 0;
   for (j = 0; j < n; j++)
   {
      if (pos[j] == -1)
      {
         index = j;
```

```
                break;
            }
            if (pos[j] > farthest)
            {
                farthest = pos[j];
                index = j;
            }
        }

        p[index] = rs[i];
    }

display:
    printf("\nAfter accessing %d:\n", rs[i]);
    for (j = 0; j < n; j++)
        printf("Frame %d: %d\n", j + 1, p[j]);
}

printf("\nTotal Page Faults = %d\n", fault);

return 0;
}
```


Output

```
Name:Gajendra
Optimal Page Replacement Algorithm

Enter number of frames: 4
Enter reference string (end with -1): 3 5 4 3 8 7 5 4 3 5 7 8 -1

After accessing 3:
Frame 1: 3
Frame 2: -1
Frame 3: -1
Frame 4: -1

After accessing 5:
Frame 1: 3
Frame 2: 5
Frame 3: -1
Frame 4: -1

After accessing 4:
Frame 1: 3
Frame 2: 5
Frame 3: 4
Frame 4: -1
```

Lab 13: Page Replacement Simulation LRU

Objectives

Simulate basic CPU algorithm

Calculate turnaround and waiting times

Algorithm:

Step 1: Start the process.

Step 2: Declare the size

Step 3: Get the number of pages to be inserted

Step 4: Get the value

Step 5: Declare counter and stack

Step 6: Select the least recently used page by counter value

Step 7: Stack them according the selection.

Step 8: Display the values

Step 9: Stop the processInput dataset of processes.

Program:

```
 #include <stdio.h>

int main()
{
   int i, j, k, n, m;
   int rs[30], p[30];
   int time[30];
   int fault = 0, flag, count = 0, index, least;

   printf("Name: Gajendra\n");
   printf("LRU Page Replacement Algorithm\n\n");

   printf("Enter number of frames: ");
   scanf("%d", &n);

   printf("Enter reference string (end with -1): ");
   i = 0;
   while (1)
   {
      scanf("%d", &rs[i]);
      if (rs[i] == -1)
```

```
        break;
    i++;
}
m = i;

/* Initialize frames and time */
for (i = 0; i < n; i++)
{
    p[i] = -1;
    time[i] = 0;
}

/* LRU logic */
for (i = 0; i < m; i++)
{
    flag = 0;
    count++;

    /* Page hit */
    for (j = 0; j < n; j++)
    {
        if (p[j] == rs[i])
        {
            time[j] = count;
            flag = 1;
            break;
        }
    }

    /* Page fault */
    if (flag == 0)
    {
        fault++;

        /* Check for empty frame */
        for (j = 0; j < n; j++)
        {
            if (p[j] == -1)
            {
                p[j] = rs[i];
                time[j] = count;
                goto display;
            }
        }

        /* Find least recently used page */
```

```
            least = time[0];
            index = 0;
            for (j = 1; j < n; j++)
            {
               if (time[j] < least)
               {
                  least = time[j];
                  index = j;
               }
            }

            p[index] = rs[i];
            time[index] = count;
         }

      display:
         printf("\nAfter accessing %d:\n", rs[i]);
         for (j = 0; j < n; j++)
            printf("Frame %d: %d\n", j + 1, p[j]);
      }

      printf("\nTotal Page Faults = %d\n", fault);

      return 0;
}
```

Output

```
Name: Gajendra
LRU Page Replacement Algorithm

Enter number of frames: 3
Enter reference string (end with -1): 5 20 13 123 4 5 3 7 3  7 2 7 -1

After accessing 5:
Frame 1: 5
Frame 2: -1
Frame 3: -1

After accessing 20:
Frame 1: 5
Frame 2: 20
Frame 3: -1

After accessing 13:
Frame 1: 5
Frame 2: 20
Frame 3: 13

After accessing 123:
Frame 1: 123
Frame 2: 20
Frame 3: 13
```

Lab 14: Page Replacement Simulation Clock

Objectives

Simulate advanced page replacement policies

Algorithm

Step 1: Start the process.

Step 2: Declare the number of frames.

Step 3: Read the number of pages in the reference string.

Step 4: Read the page reference values.

Step 5: Initialize all frames as empty and set the use (reference) bit of each frame to 0.

Step 6: Set the clock pointer to the first frame.

Step 7: For each page in the reference string, check whether the page is already present in any frame.

- If present, set its use bit to 1 (page hit).

Step 8: If the page is not present (page fault):

- Check the frame pointed by the clock pointer.
    - If the use bit is 0, replace the page with the new page.
    - If the use bit is 1, set it to 0 and move the pointer to the next frame.
- Repeat until a frame with use bit 0 is found.

Step 9: Move the clock pointer to the next frame after replacement.

Step 10: Display the frame contents after each page reference.

Step 11: Display the total number of page faults.

Step 12: Stop the process

Program

```
#include <stdio.h>

int main()
{
   int n, m, i, j;
   int frames[30], usebit[30];
   int rs[30];
   int pointer = 0;
   int fault = 0;
   int flag;

   printf("Name: Gajendra\n");
   printf("Clock (Second-Chance) Page Replacement Algorithm\n\n");

   printf("Enter number of frames: ");
   scanf("%d", &n);

   printf("Enter reference string (end with -1): ");
   i = 0;
   while (1)
   {
      scanf("%d", &rs[i]);
      if (rs[i] == -1)
         break;
      i++;
   }
   m = i;

   /* Initialize frames and use bits */
   for (i = 0; i < n; i++)
   {
      frames[i] = -1;
      usebit[i] = 0;
   }

   /* Clock algorithm */
   for (i = 0; i < m; i++)
   {
      flag = 0;

      /* Page hit check */
      for (j = 0; j < n; j++)
      {
         if (frames[j] == rs[i])
         {
```

```
            usebit[j] = 1;
            flag = 1;
            break;
        }
    }

    /* Page fault */
    if (flag == 0)
    {
        fault++;

        while (usebit[pointer] == 1)
        {
            usebit[pointer] = 0;
            pointer = (pointer + 1) % n;
        }

        frames[pointer] = rs[i];
        usebit[pointer] = 1;
        pointer = (pointer + 1) % n;
    }

    /* Display frames */
    printf("\nAfter accessing %d:\n", rs[i]);
    for (j = 0; j < n; j++)
        printf("Frame %d: %d (use=%d)\n", j + 1, frames[j], usebit[j]);
}

printf("\nTotal Page Faults = %d\n", fault);

return 0;
}
```

Output

```
Name: Gajendra
Clock (Second-Chance) Page Replacement Algorithm

Enter number of frames: 3
Enter reference string (end with -1): 1 2 3 4 5 6 7 8 9 10 11 12 13 14 -1

After accessing 1:
Frame 1: 1 (use=1)
Frame 2: -1 (use=0)
Frame 3: -1 (use=0)

After accessing 2:
Frame 1: 1 (use=1)
Frame 2: 2 (use=1)
Frame 3: -1 (use=0)

After accessing 3:
Frame 1: 1 (use=1)
Frame 2: 2 (use=1)
Frame 3: 3 (use=1)
```

Lab 15: Page Replacement Simulation using LFU

Objectives

Simulate advanced page replacement policies

Algorithm:

Step 1: Start the process

Step 2: Declare the size

Step 3: Get the number of pages to be inserted

Step 4: Get the value

Step 5: Declare counter and stack

Step 6: Select the least frequently used page by counter value

Step 7: Stack them according the selection.

Step 8: Display the values Step 9: Stop the process

Program:

```
#include <stdio.h>

int main()
{
   int f, p;
   int pages[50], frame[10], hit = 0, count[50], time[50];
   int i, j, page, flag, least, minTime, temp;
   printf("Your name : Gajendra\n");
   printf("Enter no of frames : ");
   scanf("%d", &f);
   printf("Enter no of pages : ");
   scanf("%d", &p);
   for (i = 0; i < f; i++)
   {
      frame[i] = -1;
   }
   for (i = 0; i < 50; i++)
   {
      count[i] = 0;
   }
   printf("Enter page no : \n");
   for (i = 0; i < p; i++)
   {
```

```
    scanf("%d", &pages[i]);
}
printf("\n");
for (i = 0; i < p; i++)
{
    count[pages[i]]++;
    time[pages[i]] = i;
    flag = 1;
    least = frame[0];
    for (j = 0; j < f; j++)
    {
        if (frame[j] == -1 || frame[j] == pages[i])
        {
            if (frame[j] != -1)
            {
                hit++;
            }
            flag = 0;
            frame[j] = pages[i];
            break;
        }
        if (count[least] > count[frame[j]])
        {
            least = frame[j];
        }
    }
    if (flag)
    {
        minTime = 50;
        for (j = 0; j < f; j++)
        {
            if (count[frame[j]] == count[least] && time[frame[j]] < minTime)
            {
                temp = j;
                minTime = time[frame[j]];
            }
        }
        count[frame[temp]] = 0;
        frame[temp] = pages[i];
    }
    for (j = 0; j < f; j++)
    {
        printf("%d ", frame[j]);
    }
```

```
        printf("\n");
    }
    printf("Page hit = %d", hit);
    return 0;
}
```

Output

```
Your name : Gajendra
Enter no of frames : 3
Enter no of pages : 1 2 3 4 5 6 7 8  9 10 -1
Enter page no :

2 -1 -1
Page hit = 0
```

PROVIDED BY: GAJENDRA AWASTHI

Lab 16: Disk Scheduling using FCFS

Objectives

Understand disk arm scheduling

Program

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
   int RQ[100], i, n, TotalHeadMoment = 0, initial;
   printf("Name:'Gajendra'\n");
   printf("Enter the number of Requests\n");
   scanf("%d", &n);
   printf("Enter the Requests sequence\n");
   for (i = 0; i < n; i++)
      scanf("%d", &RQ[i]);
   printf("Enter initial head position\n");
   scanf("%d", &initial);

   // logic for FCFS disk scheduling

   for (i = 0; i < n; i++)
   {
      TotalHeadMoment = TotalHeadMoment + abs(RQ[i] - initial);
      initial = RQ[i];
   }

   printf("Total head moment is %d", TotalHeadMoment);
   return 0;
}
```

Output

```
Name:'Gajendra'
Enter the number of Requests
3
Enter the Requests sequence
2 3 6
Enter initial head position
3
Total head moment is 5
```

Lab 17: Disk Scheduling using SCAN

Objectives

Understand disk arm scheduling

Program

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
   int RQ[100], i, j, n, TotalHeadMoment = 0, initial, size, move;
   printf("Name:Gajendra\n");
   printf("Enter the number of Requests\n");
   scanf("%d", &n);
   printf("Enter the Requests sequence\n");
   for (i = 0; i < n; i++)
      scanf("%d", &RQ[i]);
   printf("Enter initial head position\n");
   scanf("%d", &initial);
   printf("Enter total disk size\n");
   scanf("%d", &size);
   printf("Enter the head movement direction for high 1 and for low 0\n");
   scanf("%d", &move);

   // logic for Scan disk scheduling

   /*logic for sort the request array */
   for (i = 0; i < n; i++)
   {
      for (j = 0; j < n - i - 1; j++)
      {
         if (RQ[j] > RQ[j + 1])
         {
            int temp;
            temp = RQ[j];
            RQ[j] = RQ[j + 1];
            RQ[j + 1] = temp;
         }
      }
   }

   int index;
   for (i = 0; i < n; i++)
   {
      if (initial < RQ[i])
```

```
        {
            index = i;
            break;
        }
    }

    // if movement is towards high value
    if (move == 1)
    {
        for (i = index; i < n; i++)
        {
            TotalHeadMoment = TotalHeadMoment + abs(RQ[i] - initial);
            initial = RQ[i];
        }
        //  last movement for max size
        TotalHeadMoment = TotalHeadMoment + abs(size - RQ[i - 1] - 1);
        initial = size - 1;
        for (i = index - 1; i >= 0; i--)
        {
            TotalHeadMoment = TotalHeadMoment + abs(RQ[i] - initial);
            initial = RQ[i];
        }
    }
    // if movement is towards low value
    else
    {
        for (i = index - 1; i >= 0; i--)
        {
            TotalHeadMoment = TotalHeadMoment + abs(RQ[i] - initial);
            initial = RQ[i];
        }
        //  last movement for min size
        TotalHeadMoment = TotalHeadMoment + abs(RQ[i + 1] - 0);
        initial = 0;
        for (i = index; i < n; i++)
        {
            TotalHeadMoment = TotalHeadMoment + abs(RQ[i] - initial);
            initial = RQ[i];
        }
    }

    printf("Total head movement is %d", TotalHeadMoment);
    return 0;
}
```

Output

```
Name:Gajendra
Enter the number of Requests
4
Enter the Requests sequence
1 5 8 10
Enter initial head position
2
Enter total disk size
23
Enter the head movement direction for high 1 and for low 0
1
Total head movement is 41
```

Lab 18: Disk Scheduling using C-SCAN

Objectives

Simulate advance disk scheduling

Program

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
   int RQ[100], i, j, n, TotalHeadMoment = 0, initial, size, move;
   printf("Name:Gajendra\n");
   printf("Enter the number of Requests\n");
   scanf("%d", &n);
   printf("Enter the Requests sequence\n");
   for (i = 0; i < n; i++)
      scanf("%d", &RQ[i]);
   printf("Enter initial head position\n");
   scanf("%d", &initial);
   printf("Enter total disk size\n");
   scanf("%d", &size);
   printf("Enter the head movement direction for high 1 and for low 0\n");
   scanf("%d", &move);

   // logic for C-Scan disk scheduling

   /*logic for sort the request array */
   for (i = 0; i < n; i++)
   {
      for (j = 0; j < n - i - 1; j++)
      {
         if (RQ[j] > RQ[j + 1])
         {
            int temp;
            temp = RQ[j];
            RQ[j] = RQ[j + 1];
            RQ[j + 1] = temp;
         }
      }
   }

   int index;
   for (i = 0; i < n; i++)
   {
      if (initial < RQ[i])
```

```
    {
        index = i;
        break;
    }
}

// if movement is towards high value
if (move == 1)
{
    for (i = index; i < n; i++)
    {
        TotalHeadMoment = TotalHeadMoment + abs(RQ[i] - initial);
        initial = RQ[i];
    }
    //  last movement for max size
    TotalHeadMoment = TotalHeadMoment + abs(size - RQ[i - 1] - 1);
    /*movement max to min disk */
    TotalHeadMoment = TotalHeadMoment + abs(size - 1 - 0);
    initial = 0;
    for (i = 0; i < index; i++)
    {
        TotalHeadMoment = TotalHeadMoment + abs(RQ[i] - initial);
        initial = RQ[i];
    }
}
// if movement is towards low value
else
{
    for (i = index - 1; i >= 0; i--)
    {
        TotalHeadMoment = TotalHeadMoment + abs(RQ[i] - initial);
        initial = RQ[i];
    }
    //  last movement for min size
    TotalHeadMoment = TotalHeadMoment + abs(RQ[i + 1] - 0);
    /*movement min to max disk */
    TotalHeadMoment = TotalHeadMoment + abs(size - 1 - 0);
    initial = size - 1;
    for (i = n - 1; i >= index; i--)
    {
        TotalHeadMoment = TotalHeadMoment + abs(RQ[i] - initial);
        initial = RQ[i];
    }
}
```

```
    printf("Total head movement is %d", TotalHeadMoment);
    return 0;
}
```

Output

```
Name:Gajendra
Enter the number of Requests
3
Enter the Requests sequence
1 46 6
Enter initial head position
3
Enter total disk size
34
Enter the head movement direction for high 1 and for low 0
1
Total head movement is 90
```

Lab 19: Simulate Banker's Algorithm for Deadlock Avoidance

Objectives

Simulate advance disk scheduling

Program:

```
#include <stdio.h>
// #include<conio.h>
void main()
{
   int available[3], work[5], max[5][3], allocation[5][3], need[5][3], safe[5], totalres[5];
   char finish[5];
   int i, j, k, totalloc = 0, state, value = 0;
   // clrscr();
   printf("Name:Gajendra\n");

   printf("Enter Instances of each Resource : ");
   for (i = 0; i < 3; i++)
   {
      scanf("%d", &totalres[i]);
   }
   printf("Enter Maximum resources for each processes: ");
   for (i = 0; i < 5; i++)
   {
      for (j = 0; j < 3; j++)
      {
         printf("\n Enter process %d Resource %d : ", i, (j + 1));
         scanf("%d", &max[i][j]);
      }
   }
   // clrscr();
   printf("Enter number of resources allocated to each Process");
   for (i = 0; i < 5; i++)
   {
      for (j = 0; j < 3; j++)
      {
         printf("\n Enter the resource of R%d allocated to process %d : ", (j + 1), i);
         scanf("%d", &allocation[i][j]);
      }
   }
   for (i = 0; i < 5; i++)
   {
      for (j = 0; j < 3; j++)
      {
         need[i][j] = max[i][j] - allocation[i][j];
```

```
    }
}
for (i = 0; i < 5; i++)
{
    finish[i] = 'f';
}
for (i = 0; i < 3; i++)
{
    totalloc = 0;
    for (j = 0; j < 5; j++)
    {
        totalloc = totalloc + allocation[j][i];
    }
    available[i] = totalres[i] - totalloc;
    work[i] = available[i];
}
// clrscr();
printf("\n Allocated Resources \n");
for (i = 0; i < 5; i++)
{
    for (j = 0; j < 3; j++)
    {
        printf("%d", allocation[i][j]);
    }
    printf("\n");
}
printf("\n Maximum Resources \n");
for (i = 0; i < 5; i++)
{
    for (j = 0; j < 3; j++)
    {
        printf("%d", max[i][j]);
    }
    printf("\n");
}
printf("\n Needed Reources \n");
for (i = 0; i < 5; i++)
{
    for (j = 0; j < 3; j++)
    {
        printf("%d", need[i][j]);
    }
    printf("\n");
}
printf("\n Available Reources");
```

```
for (i = 0; i < 3; i++)
{
   printf("%d", available[i]);
}
printf("\n");
for (i = 0; i < 5; i++)
{
   for (j = 0; j < 3; j++)
   {
      if ((finish[i] == 'f') && (need[i][j] <= work[j]))
      {
         state = 1;
         continue;
      }
      else
      {
         state = 0;
         break;
      }
   }
   if (state == 1)
   {
      for (j = 0; j < 3; j++)
      {
         work[j] = work[j] + allocation[i][j];
      }
      finish[i] = 't';
      safe[value] = i;
      ++value;
   }
   if (i == 4)
   {
      if (value == 5)
      {
         break;
      }
      else
      {
         i = -1;
      }
   }
}
printf("\n Safe States are");
for (i = 0; i < 5; i++)
{
```

```
        printf("P%d", safe[i]);
    }
}
```

Output:

```
Enter Instances of each Resource : 10
5
7
Enter Maximum resources for each processes:
 Enter process 0 Resource 1 : 7

 Enter process 0 Resource 2 : 5

 Enter process 0 Resource 3 : 4

 Enter process 1 Resource 1 : 6

 Enter process 1 Resource 2 : 7
```

Lab 20: Simulate Banker's Algorithm for Deadlock Prevention

Objectives:

Simulate advance disk scheduling

Program:

```
#include <stdio.h>
// #include<conio.h>
void main()
{
   int nort, nopro, avail[20], req[20][20], i, j, k, flag = 0;
   //  clrscr();
   printf("Name:Gajju\n");
   printf("\n Enter the no of resource types: ");
   scanf("%d", &nort);

   printf("\n Enter the no of instances of each resource type:");
   for (i = 0; i < nort; i++)
      scanf("%d", &avail[i]);

   printf("\n Enter the no of processes:");
   scanf("%d", &nopro);

   printf("\n Enter the requests of each process:");
   for (i = 0; i < nopro; i++)
      for (j = 0; j < nort; j++)
         scanf("%d", &req[i][j]);

   for (i = 0; i < nopro; i++)
   {
      flag = 0;
      for (j = 0; j < nort; j++)
      {
         if (req[i][j] > avail[j])
         {
            flag = 1;
         }
      }
      if (flag == 1)
      {
         printf("\n resources for process p%d cannot be allocated to prevent deadlock", i);
      }
      else
      {
         for (k = 0; k < nort; k++)
```

```
            {
               avail[k] = avail[k] - req[i][k];
               printf("\n%d instances of resource type R%d are allocated to process P%d", req[i][k],
k, i);
            }
         }
      }
      printf("\n remaining resources after allocation are");
      for (i = 0; i < nort; i++)
         printf("\n  %d", avail[i]);
      //  getch();
}
```

Output:

```
Name:Gajju

 Enter the no of resource types: 2

 Enter the no of instances of each resource type:3
2 3 4 6 7 8 3 2 4 5 6 7 -1

 Enter the no of processes:
 Enter the requests of each process:
 resources for process p0 cannot be allocated to prevent deadlock
 resources for process p1 cannot be allocated to prevent deadlock
3 instances of resource type R0 are allocated to process P2
2 instances of resource type R1 are allocated to process P2
 remaining resources after allocation are
  0
  0

...Program finished with exit code 2
Press ENTER to exit console.
```